

AI-Based Smart Wearable Alert System

For Hearing-Impaired Users

Project Documentation & Technical Report

Project Type	AI / Machine Learning - Sound Classification
Technology Stack	Python 3.10, TensorFlow/Keras, Librosa, Sounddevice
Core Feature	Real-time environmental sound detection & alerts
Logging	Google Sheets (via Apps Script API)
Dashboard	HTML-based Sound Alert Dashboard

1. Project Overview

This project is a smart wearable alert system designed to assist hearing-impaired individuals in becoming aware of important environmental sounds. A microphone captures surrounding audio, an AI model classifies the sound in real time, and the user is alerted via visual messages or vibration - all without requiring cloud connectivity.

Problem it solves:

- Hearing-impaired users miss critical sounds: alarms, doorbells, someone calling them.
- Traditional solutions need multiple sensors installed throughout a building - expensive and inflexible.
- This system uses a single microphone + AI to handle all sound types portably.

System Workflow:

Environmental Sound -> Microphone -> Edge Processing (Python) -> AI Classification -> Event Detection
-> Alert Generation -> Event Logging

2. What I Built - Step by Step

Step
1

Dataset Preparation

Collected audio recordings of three target sounds - **clap**, **doorbell**, and **alarm**. Each sound was stored in its own folder under the `data/` directory as `.wav` files.

Step
2

Feature Extraction (`extract_features.py`)

Used the **Librosa** library to extract MFCC (Mel Frequency Cepstral Coefficients) features from each audio file. MFCCs capture the important frequency characteristics of sound. The resulting feature arrays were saved as `X.npy` (features) and `y.npy` (labels).

Step
3

Model Training (`train_model.py`)

Built a neural network using **TensorFlow/Keras** with Dense and Dropout layers. Trained for 50 epochs on the extracted features. The trained model was saved to `saved_model/my_model.h5`.

Step
4

Sound Prediction (`predict.py` / `predict2.py`)

Loaded the saved model and ran inference on audio files. The model outputs probability scores for each sound class and prints the predicted sound (e.g., 'Predicted sound: clap').

Step
5

Audio Recording (`record_audio.py`)

Recorded live audio from the laptop microphone using **sounddevice** and saved it as a `.wav` file for feeding into the prediction pipeline.

Step
6

Alert & Logging System

On sound detection, a visual alert is triggered and the event (timestamp, sound type, alert type) is logged to a **Google Sheet** via a Google Apps Script web endpoint (HTTP POST).

Step
7

Dashboard (`sound_dashboard.html`)

Built a simple HTML dashboard that displays the logged sound alert events in a table - showing Timestamp, Sound, and Alert columns.

3. Source Code

extract_features.py - Extract MFCC features from audio dataset

```
import os, numpy as np, librosa

DATA_PATH = "data"
features, labels = [], []

for label, folder in enumerate(os.listdir(DATA_PATH)):
    folder_path = os.path.join(DATA_PATH, folder)
    for file in os.listdir(folder_path):
        file_path = os.path.join(folder_path, file)
        audio, sr = librosa.load(file_path, sr=None)
        mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
        mfcc_mean = np.mean(mfcc.T, axis=0)
        features.append(mfcc_mean)
        labels.append(label)

X, y = np.array(features), np.array(labels)
np.save("X.npy", X)
np.save("y.npy", y)
print("Features extracted successfully!")
```

train_model.py - Train the neural network

```
import numpy as np, tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

X = np.load("X.npy")
y = tf.keras.utils.to_categorical(np.load("y.npy"))

model = Sequential([
    Dense(128, activation='relu', input_shape=(X.shape[1],)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(y.shape[1], activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=50, batch_size=16, validation_split=0.2)
model.save("saved_model/my_model.h5")
```

predict.py - Load model and predict sound from an audio file

```
import librosa, numpy as np
from tensorflow.keras.models import load_model

model = load_model("saved_model/my_model.h5")
audio_file = "data/alarm/alarm1.wav"

audio, sr = librosa.load(audio_file, sr=None)
mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
mfcc_mean = np.mean(mfcc.T, axis=0).reshape(1, -1)
```

```
prediction = model.predict(mfcc_mean)
classes = ["clap", "doorbell", "alarm"]
print("Prediction probabilities:", prediction)
print("Predicted sound:", classes[np.argmax(prediction)])
```

record_audio.py - Record live audio from microphone

```
import sounddevice as sd, wavio

duration = 5 # seconds
sample_rate = 44100
device_id = 7 # microphone device ID

print("Recording...")
audio = sd.rec(int(duration * sample_rate),
               samplerate=sample_rate, channels=1, device=device_id)
sd.wait()

wavio.write("recorded_audio.wav", audio, sample_rate, sampwidth=2)
print("Audio saved as recorded_audio.wav")
```

4. Project Screenshots

Step 1 - Environment Setup & Model Loading (Command Prompt)

Installed required Python packages (sounddevice, numpy) using pip. Navigated to the project directory and ran load_model.py. TensorFlow loaded the trained model successfully - confirmed by the 'Loading AI sound model...' message.

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\GSSS>py -3.10 -m pip install sounddevice
Collecting sounddevice
  Downloading sounddevice-0.5.5-py3-none-win_amd64.whl (365 kB)
-----
Collecting cffi
  Downloading cffi-2.0.0-cp310-cp310-win_amd64.whl (182 kB)
-----
Collecting pycparser
  Downloading pycparser-3.0-py3-none-any.whl (48 kB)
-----
Installing collected packages: pycparser, cffi, sounddevice
Successfully installed cffi-2.0.0 pycparser-3.0 sounddevice-0.5.5

[notice] A new release of pip is available: 23.0.1 -> 24.0.1
[notice] To update, run: C:\Users\GSSS\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip

C:\Users\GSSS>py -3.10 -m pip install numpy
Requirement already satisfied: numpy in c:\users\gsss\appdata\local\programs\python\python310\lib\site-packages (2.2.6)

[notice] A new release of pip is available: 23.0.1 -> 24.0.1
[notice] To update, run: C:\Users\GSSS\AppData\Local\Programs\Python\Python310\python.exe -m pip install --upgrade pip

C:\Users\GSSS>py -3.10 load_model.py
C:\Users\GSSS\AppData\Local\Programs\Python\Python310\python.exe: can't open file 'C:\Users\GSSS\load_model.py': [Errno 2] No such file or directory

C:\Users\GSSS>cd Desktop
C:\Users\GSSS\Desktop>cd AI_Alert_System
C:\Users\GSSS\Desktop\AI_Alert_System>cd code
C:\Users\GSSS\Desktop\AI_Alert_System\code>py -3.10 load_model.py
2026-03-14 14:48:57.3456051: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 14:48:57.918019: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING:tensorflow:From C:\Users\GSSS\AppData\Local\Programs\Python\Python310\lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

Loading AI sound model...
WARNING:tensorflow:From C:\Users\GSSS\AppData\Local\Programs\Python\Python310\lib\site-packages\tensorflow_hub\resolver.py:120: The name tf.gfile.MakeDirs is deprecated. Please use tf.io.gfile.makedirs instead.
WARNING:tensorflow:From C:\Users\GSSS\AppData\Local\Programs\Python\Python310\lib\site-packages\tensorflow_hub\resolver.py:120: The name tf.gfile.MakeDirs is deprecated. Please use tf.io.gfile.makedirs instead.
WARNING:tensorflow:From C:\Users\GSSS\AppData\Local\Programs\Python\Python310\lib\site-packages\tensorflow_hub\module_v2.py:126: The name tf.saved_model.load_v2 is deprecated. Please use tf.compat.v2.saved_model.load instead.
```

Figure 1: Terminal showing package installation and model loading via load_model.py

Step 2 - Running predict2.py (Single Prediction)

Ran predict2.py to classify a live or pre-recorded audio sample. The model processed the MFCC features and returned a prediction. Output: **Predicted sound: clap**

```
C:\Users\GSSS\Desktop\AI_Alert_System\code>python predict2.py
2026-03-14 14:48:41.684421: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 14:48:43.939908: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 14:48:45.550294: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
+1m1/1-1[0m +132m
+1m1/1-1[0m +132m
Predicted sound: clap
```

Figure 2: Terminal output showing 'Predicted sound: clap' after running predict2.py

Step 3 - Multiple Predictions (doorbell, clap)

Ran predict2.py multiple times to test different audio samples. The model correctly identified **doorbell** and **clap** sounds across separate runs, with prediction probabilities also printed.

```
C:\Users\GSSS\Desktop\AI_Alert_System\code>python predict2.py
2026-03-14 11:48:56.857674: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:48:58.729387: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:48:59.895764: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA4, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
~[1m]1~[0m ~[32m~[0m~[37m~[0m ~[1m0s~[0m 57ms/step
Predicted sound: doorbell

C:\Users\GSSS\Desktop\AI_Alert_System\code>python predict2.py
2026-03-14 11:45:40.585782: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:45:42.413582: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:45:43.548285: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA4, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
~[1m]1~[0m ~[32m~[0m~[37m~[0m ~[1m0s~[0m 57ms/step
Predicted sound: clap

C:\Users\GSSS\Desktop\AI_Alert_System\code>python predict2.py
2026-03-14 11:47:13.171434: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:47:15.420812: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-14 11:47:16.678569: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA4, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
~[1m]1~[0m ~[32m~[0m~[37m~[0m ~[1m0s~[0m 57ms/step
Prediction probabilities: [[1.0000000e+00 1.5228281e-23 1.7122027e-23]]
Predicted sound: clap
```

Figure 3: Multiple prediction runs - correctly classifying doorbell and clap sounds

Step 4 - Event Logging to Google Sheets

Detected sound events were automatically logged to a Google Sheet via the Apps Script API endpoint. Each row records the Timestamp, Sound type, and Alert type (Visual Alert).

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Timestamp	Sound	Alert											
2	2026-03-14 14:4	clap	Visual Alert											
3	2026-03-14 15:0	clap	Visual Alert											
4														
5														

Figure 4: Google Sheets log showing timestamped clap detections with 'Visual Alert' entries

Step 5 - HTML Alert Dashboard

An HTML dashboard (sound_dashboard.html) was built to display logged events. It renders a table with Timestamp, Sound, and Alert columns - providing a simple visual interface for reviewing detections.

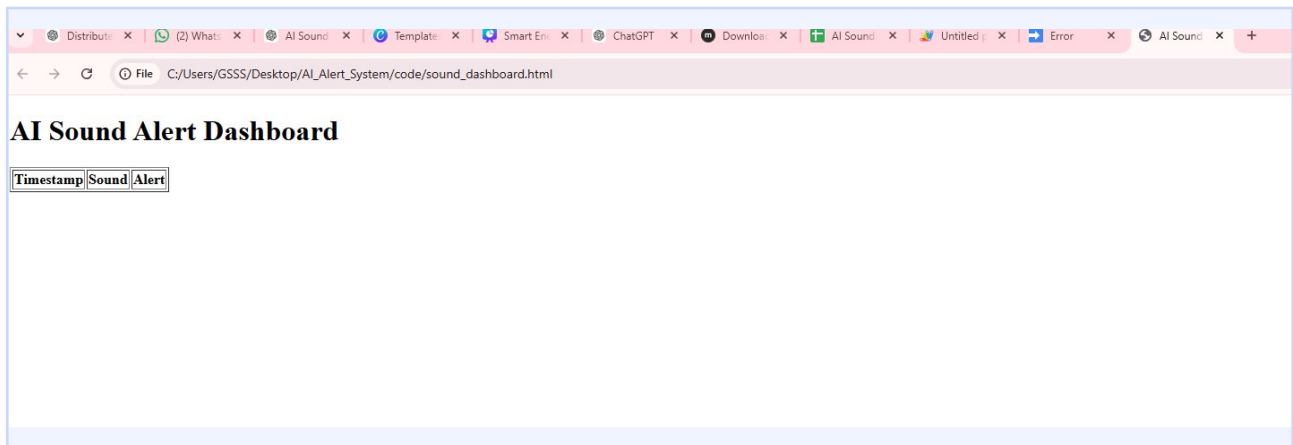


Figure 5: AI Sound Alert Dashboard rendered in Chrome - showing the log table headers

5. Project File Structure

File / Folder	Purpose
data/clap/	Audio training samples for 'clap' sound
data/alarm/	Audio training samples for 'alarm' sound
data/doorbell/	Audio training samples for 'doorbell' sound
extract_features.py	Loads audio files, extracts MFCC features, saves X.npy & y.npy
train_model.py	Trains the neural network on extracted features, saves model
saved_model/my_model.h5	The trained Keras model (used for predictions)
predict.py	Loads model, classifies a given .wav audio file
predict2.py	Extended predict script with probability output
record_audio.py	Records audio from microphone using sounddevice
load_model.py	Loads and verifies the saved model
sound_dashboard.html	HTML dashboard to view logged alert events
X.npy	Extracted MFCC feature arrays (input to model)
y.npy	Corresponding sound category labels

6. External Integrations

Google Apps Script Logging Endpoint

Sound detection events are sent via HTTP POST to a Google Apps Script web app, which appends the event to a Google Sheet. This provides persistent logging without requiring a backend server.

```
Endpoint: https://script.google.com/macros/s/AKfycbxWMxDqok_p7x1LBwQCYQ  
BPdzkkEJqw8gV2cE45qbkiFJlD0Szzsfy8H-QscMYneThg2Q/exec
```

```
Payload (POST): { timestamp, sound, alert }
```

Logged Fields:

- Timestamp - date and time of detection
- Sound - predicted sound class (clap / doorbell / alarm)
- Alert - type of alert triggered (e.g., Visual Alert)

7. Setup & Run Instructions

1

Install Python 3.10

Download from python.org and install. Ensure it is added to PATH.

2

Install dependencies

Run: `py -3.10 -m pip install tensorflow librosa sounddevice numpy wavio`

3

Prepare dataset

Place .wav audio files in `data/clap/`, `data/alarm/`, `data/doorbell/`

4

Extract features

Run: `py -3.10 extract_features.py` - generates `X.npy` and `y.npy`

5

Train the model

Run: `py -3.10 train_model.py` - saves model to `saved_model/my_model.h5`

6

Predict a sound

Run: `py -3.10 predict.py` - outputs predicted sound class to terminal

7

Open dashboard

Open `sound_dashboard.html` in a browser to view logged events

8. Future Improvements

- Smartwatch or wristband integration for haptic (vibration) alerts
- Mobile push notifications via a companion app
- Custom user-trained sound models (teach the system new sounds)
- Improved edge AI optimization for lower-power microcontrollers
- Expanded sound vocabulary beyond the current 3 classes
- Real-time streaming prediction instead of file-based batch inference